

SWIFT SHIP TRACKER

SALESFORCE-BASED PARCEL TRACKING AND AGENTFORCE AI SYSTEM

Sree Sastha Institute of Engineering and Technology

Team ID : SWTID-2026-5977

Team Size : 5

Team Leader : Karthiga B

emailid: karthiga.b613@gmail.com

Team member : Kalaiyarasi T

emailid: kalaiyarasikalaiyarasi171@gmail.com

Team member : Rubini M

emailid: rubinim68@gmail.com

Team member : Deva Dharshini K

emailid: devadharshinipriya982@gmail.com

Team member : Priya B

emailid: priyapriya62895@gmail.com

SWIFT SHIP TRACKER

SALESFORCE-BASED PARCEL TRACKING AND AGENTFORCE AI SYSTEM

PROJECT DOCUMENTATION

ABSTRACT

SwiftShip Tracker is a Salesforce-based parcel tracking and delivery management solution designed to centralize parcel-related information and provide a conversational AI tracking experience. The system uses Salesforce custom objects for Parcel, Delivery, Sender and Receiver information, with relationships and fields designed around parcel identification, status, weight, estimated delivery and contact information.

Salesforce Flow is used to retrieve a Parcel record from a Parcel ID. Prompt Builder is used to structure parcel-tracking information, while Agentforce provides a conversational interface through which users can request tracking information. The current implementation also includes custom tabs, a SwiftShip Tracker Lightning application, a Swift Ship permission set, sample Apex data and Salesforce CLI deployment scripts.

The project is maintained in GitHub as a Salesforce DX source project. The current implementation is intended as an academic/prototype solution in a Salesforce Developer Org. Advanced capabilities such as production UAT, enterprise sandbox staging, CI/CD, email alerts, reports/dashboards and Experience Cloud are documented as future extensions rather than claimed as implemented features.

TABLE OF CONTENTS

1. Introduction
2. Ideation Phase
3. Requirement Analysis Phase
4. Project Design Phase
5. Project Planning & Scheduling
6. Project Development Phase
 - 6.1 Phase 1 - Requirement Analysis & Planning
 - 6.2 Phase 2 - Salesforce Development - Backend & Configurations
 - 6.3 Milestone 1 - Salesforce Developer Org
 - 6.4 Milestone 2 - Custom Objects
 - 6.5 Milestone 3 - Custom Tabs & Lightning App
 - 6.6 Milestone 4 - Fields & Relationships
 - 6.7 Milestone 5 - Prompt Builder
 - 6.8 Milestone 6 - Parcel Details Flow
 - 6.9 Milestone 7 - Agentforce Parcel Tracking
 - 6.10 Source Control & Deployment
 - 6.11 Phase 4 - Data Migration, Testing & Security
 - 6.12 Phase 5 - Deployment, Documentation & Maintenance
7. Functional & Performance Testing
8. Advantages & Disadvantages
9. Conclusion
10. Future Scope
11. Appendix
12. References

1. INTRODUCTION

1.1 Project Overview

SwiftShip Tracker is a Salesforce CRM solution developed to simplify parcel booking, shipment tracking and delivery management. The implemented core focuses on centralized parcel data, automated record retrieval and conversational AI-assisted tracking.

1.2 Purpose

- Centralize parcel and delivery information.
- Simplify parcel tracking using Parcel ID.
- Automate parcel-detail retrieval through Salesforce Flow.
- Provide conversational parcel tracking with Agentforce.
- Maintain deployable metadata using Salesforce CLI and GitHub.

1.3 User Story

As a customer or sender, I want to track my parcel using a Parcel ID so that I can access the latest available parcel information without depending entirely on manual support. As a delivery-related user, I want structured parcel records so that shipment information can be managed consistently.

1.4 Project Scope

- Parcel, Delivery, Sender and Receiver custom objects.
- Custom fields and lookup relationships.
- Custom tabs and SwiftShip Tracker Lightning application.
- Parcel Details auto-launched Flow.
- Prompt Builder configuration for parcel details.
- Agentforce parcel tracking experience.
- Swift Ship permission set and Apex sample-data script.
- Salesforce CLI deployment and GitHub source control.

2. IDEATION PHASE

2.1 Problem Statement

PS	User	I am trying to	But	Because	Need
PS-1	Customer / Sender	book and manage a parcel	information may be spread across processes	operations need centralization	simple workflow
PS-2	Customer / Sender	track a parcel	tracking information may require manual lookup	shipment data is not always immediately accessible	quick visibility
PS-3	Delivery Agent	update parcel status	manual updates can be repetitive	operational data needs structure	efficient processing

PS-4	Customer	receive parcel information	support may be needed for routine queries	communication can be fragmented	self-service
PS-5	Admin	monitor parcel operations	data is harder to inspect when distributed	operational information needs a common platform	centralized visibility

2.2 Empathy Map - Customer / Sender

Think & Feel: Wants a simple way to access parcel information.

Hear: Receives delivery information and status updates.

See: Parcel ID, status, weight, estimated delivery and delivery details.

Say & Do: Books parcels, checks tracking information and requests updates.

Pain: Delays, confusion and dependence on support.

Gain: Self-service tracking and centralized shipment information.

2.3 Brainstorming & Idea Prioritization

3. REQUIREMENT ANALYSIS PHASE

3.1 Customer Journey

- Parcel Booking
- Parcel Record Creation
- Sender / Receiver Information
- Dispatch & Shipment
- In-Transit Tracking
- Out for Delivery
- Delivery Confirmation
- Customer Tracking / Agentforce
- Operational Monitoring

3.2 Functional Requirements

FR	Requirement	Current Status
FR-1	Parcel record creation and storage	Implemented
FR-2	Parcel ID auto-number P-{000}	Implemented
FR-3	Parcel status tracking	Implemented
FR-4	Weight management	Implemented
FR-5	Estimated delivery date	Implemented
FR-6	Sender information management	Implemented
FR-7	Receiver information management	Implemented

FR-8	Delivery and current-location information	Implemented
FR-9	Custom tabs and Lightning application	Implemented
FR-10	Parcel Details auto-launched Flow	Implemented
FR-11	Prompt Builder parcel-details prompt	Implemented
FR-12	Agentforce parcel tracking	Implemented
FR-13	Swift Ship permission set	Implemented
FR-14	Apex sample-data seeding	Implemented
FR-15	Reports and dashboards	Future / not in current package
FR-16	Experience Cloud customer portal	Future / not in current package
FR-17	Email alerts and templates	Future / not in current package

3.3 Non-Functional Requirements

NFR	Description
Usability	Simple Salesforce and conversational tracking experience.
Security	Controlled access through Salesforce permissions and configuration.
Reliability	Parcel data is retrieved from Salesforce records.
Performance	Tracking requests should return promptly within the Developer Org test environment.
Availability	Cloud-based Salesforce access.
Scalability	Data model can be extended with more logistics features.
Maintainability	Metadata, scripts and documentation are source controlled.

3.4 Data Flow Diagram

SwiftShip Tracker - Parcel Tracking Flowchart

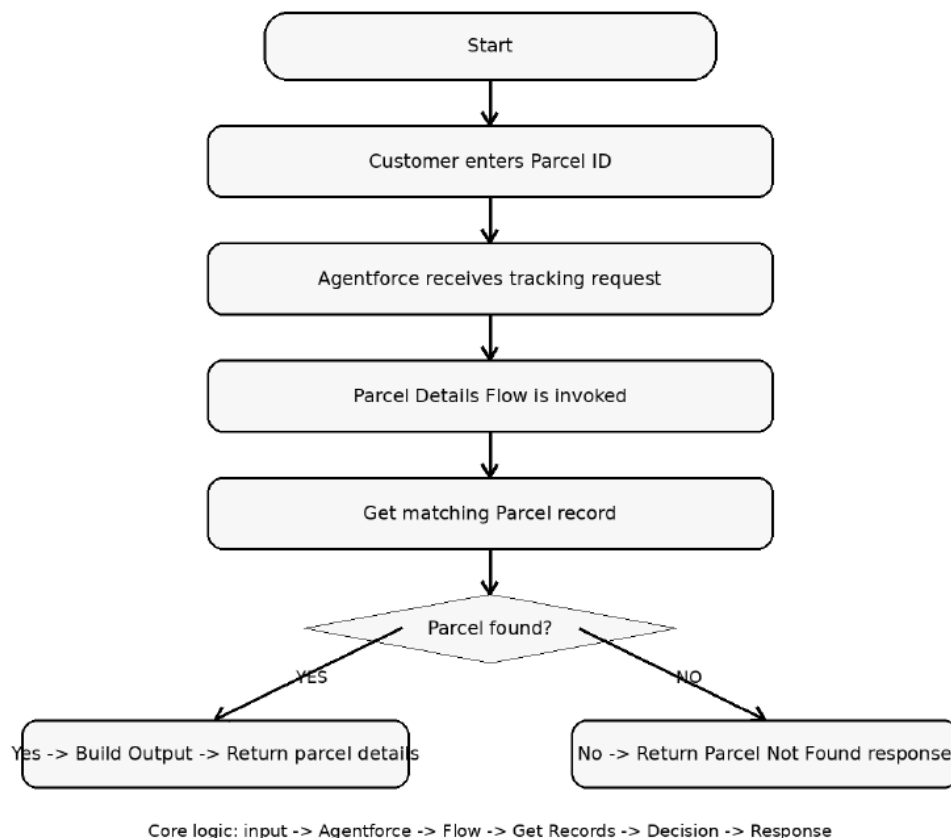


Figure 3.1 - SwiftShip Tracker parcel tracking flow.

Core logic: input -> Agentforce -> Flow -> Get Records -> Decision -> Response

Figure 3.1 - SwiftShip Tracker parcel tracking flow.

4. PROJECT DESIGN PHASE

4.1 Problem-Solution Fit

Problem	Solution	Benefit
Scattered parcel information	Parcel, Delivery, Sender and Receiver objects	Structured CRM data
Tracking queries require lookup	Agentforce + Parcel Details Flow	Conversational tracking
Manual retrieval	Auto-launched Flow	Repeatable automation
Access differs by function	Swift Ship permission set	Controlled access
Project needs repeatable	Salesforce CLI + GitHub	Version-controlled metadata

deployment		
------------	--	--

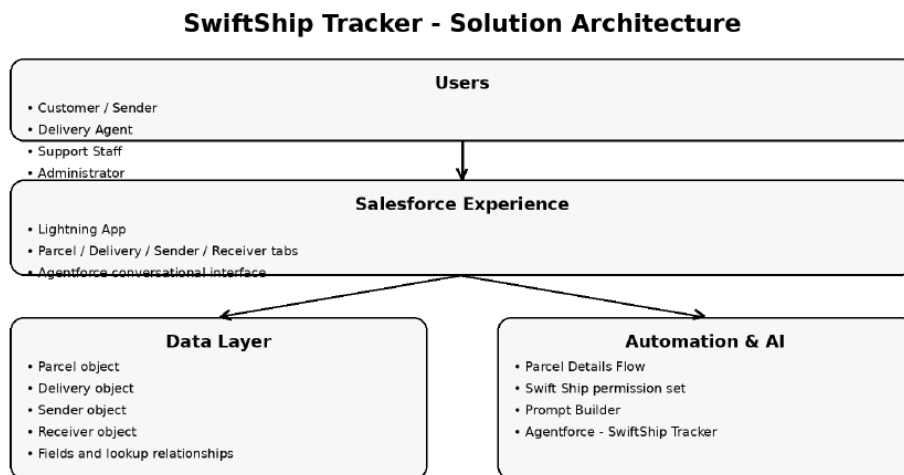
4.2 Proposed Solution

Parameter	Description
Problem	Difficult or inefficient access to parcel tracking information.
Solution	Salesforce-based parcel management and tracking workflow.
Innovation	Agentforce AI + Prompt Builder + Salesforce Flow.
Impact	Centralized parcel information and automated retrieval.
Business Model	Customer, delivery agent, support and administrator operations.
Scalability	Can be extended with notifications, analytics, portal and integrations.

4.3 Solution Architecture

Problem	Difficult or inefficient access to parcel tracking information.
Solution	Salesforce-based parcel management and tracking workflow.
Innovation	Agentforce AI + Prompt Builder + Salesforce Flow.
Impact	Centralized parcel information and automated retrieval.
Business Model	Customer, delivery agent, support and administrator operations.
Scalability	Can be extended with notifications, analytics, portal and integrations.

Solution Architecture



GitHub + Salesforce CLI provide version control and repeatable metadata deployment.

Figure 4.1 - SwiftShip Tracker solution architecture.

GitHub + Salesforce CLI provide version control and repeatable metadata deployment.

Figure 4.1 - SwiftShip Tracker solution architecture.

The architecture shows the implemented core: users access the Salesforce experience, data is stored in Parcel / Delivery / Sender / Receiver objects, and automation and AI components provide parcel retrieval and conversational responses.

4.4 Security Model - Planned / Configurable

- Administrator - full administrative access. Delivery Agent - read/edit access to operational parcel and delivery records.
- Customer - restricted access to relevant parcel information through a customer-facing interface when Experience Cloud is implemented.
- Support Staff - access needed for case and parcel issue handling.
- Field-level and record-level controls can be strengthened as the solution moves toward production.

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

- Agile-style milestone execution
- Feature-wise implementation
- Testing after configuration
- Git-based source control
- CLI-based deployment

5.2 Product Backlog

Sprint	Epic	User Story / Task	Points	Priority
Sprint-1	Developer Setup	Configure Salesforce Developer Org, CLI and Git repository.	3	High
Sprint-2	Data Modeling	Create Parcel, Delivery, Sender and Receiver objects and relationships.	5	High
Sprint-2	Navigation	Create tabs and SwiftShip Tracker Lightning application.	5	High
Sprint-3	Automation	Create Parcel Details Flow.	5	High
Sprint-4	AI	Configure Prompt Builder and Agentforce parcel tracking.	5	High
Sprint-5	Deployment	Assign permission set and seed sample data.	4	High
Sprint-6	Documentation	Capture evidence, update README	4	Medium

		and prepare final report.		
--	--	---------------------------	--	--

5.3 Project Tracker

Milestone	Deliverable	Status
M1	Salesforce Developer Org + CLI	Completed
M2	Parcel / Delivery / Sender / Receiver objects	Completed
M3	Custom tabs + Lightning app	Completed
M4	Fields and relationships	Completed
M5	Prompt Builder	Completed
M6	Parcel Details Flow	Completed
M7	Agentforce tracking agent	Completed
M8	GitHub + README + screenshots	Completed

6. PROJECT DEVELOPMENT PHASE

6.1 Phase 1 - Requirement Analysis & Planning

The development started by identifying the need for centralized parcel information and easier tracking. The project design was then translated into Salesforce metadata, automation and AI components. The source-controlled project is organized as a Salesforce DX project and supported by CLI deployment scripts.

6.2 Phase 2 - Salesforce Development - Backend & Configurations

The implementation phase focuses on the core Salesforce configuration: custom objects and fields, tabs, the Lightning application, the Parcel Details Flow, Prompt Builder, Agentforce, a permission set and sample-data script.

Implemented Feature Summary

Feature	Implementation
Custom Objects	Parcel, Delivery, Sender, Receiver
Fields	Auto Number, Picklist, Number, Date, Geolocation, Phone, Email, Lookup
Tabs	Custom tabs for project modules
Lightning App	SwiftShip Tracker
Flow	Parcel Details - Auto-Launched
Prompt	Retrieve Parcel Details
AI	Agentforce parcel-tracking agent
Access	Swift Ship permission set
Sample Data	scripts/apex/seed.apex
Version Control	Git + GitHub

6.3 Milestone 1 - Salesforce Developer Org

Salesforce Developer Edition is used as the project development and testing environment. Salesforce CLI is used to authenticate, deploy metadata, assign the permission set, execute the sample-data script and open the org.

Typical setup commands:

```
node -v
```

```
npm install -g @salesforce/cli
```

```
sf --version
```

```
sf org login web --alias swiftship --set-default
```

6.4 Milestone 2 - Custom Objects

6.5 Milestone 3 - Custom Tabs & Lightning App

Custom tabs make the project modules easy to navigate. The project metadata includes custom tabs for the parcel-related objects and the SwiftShip Tracker Lightning application.

Module	Navigation Component
Parcel	Custom Object Tab
Delivery	Custom Object Tab
Sender	Custom Object Tab
Receiver	Custom Object Tab
SwiftShip Tracker	Lightning App

6.6 Milestone 4 - Fields & Relationships

Parcel Object Fields

Field	API Name	Data Type	Purpose
Parcel ID	Parcel_ID__c	Auto Number	P-{000} unique parcel identifier
Status	Status__c	Picklist	Parcel lifecycle state
Weight	Weight__c	Number	Parcel weight
Estimated Delivery	Estimated_Delivery_Date__c	Date	Expected delivery date
Sender	Sender__c	Lookup	Sender relationship
Receiver	Receiver__c	Lookup	Receiver relationship

Delivery Object Fields

Field	API Name	Data Type	Purpose
Current Location	Current_Location__c	Geolocation	Location coordinates
Estimated Delivery	Estimated_Delivery_Date__c	Date	Expected delivery date
Sender	Sender__c	Lookup	Sender relationship
Parcel	Parcel__c	Lookup	Parcel relationship

Sender and Receiver Fields

Object	Field	Data Type
Sender	Sender_Address__c	Geolocation
Sender	Sender_Contact__c	Phone
Sender	Sender_Email__c	Email
Receiver	Receiver_Address__c	Geolocation
Receiver	Receiver_Contact__c	Phone
Receiver	Receiver_Email__c	Email
Receiver	Sender__c / Parcel__c	Lookup relationships

6.7 Milestone 5 - Prompt Builder

Prompt Builder is used to structure the parcel tracking response. The prompt references the Parcel record and relevant fields so that the output can present parcel information consistently.

- Parcel Name / record information
- Parcel ID
- Status
- Weight
- Estimated Delivery Date

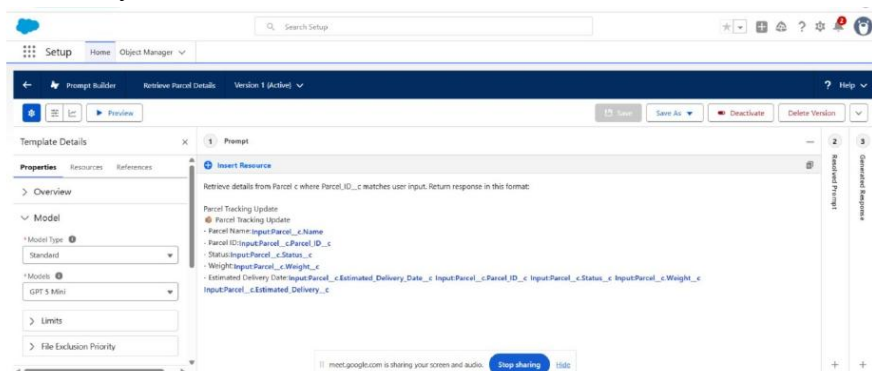


Figure 6.7.1 - Retrieve Parcel Details prompt in Prompt Builder.

6.8 Milestone 6 - Parcel Details Flow

The Parcel Details Flow is an auto-launched Flow. It receives the input variable ids, finds the matching Parcel record, evaluates whether a record was found, and prepares the Output returned to the calling agent.

Flow Element	Role
Start	Starts the auto-launched flow.

Figure 6.7.1 - Retrieve Parcel Details prompt in Prompt Builder.

6.8 Milestone 6 - Parcel Details Flow

The Parcel Details Flow is an auto-launched Flow. It receives the input variable ids, finds the matching Parcel record, evaluates whether a record was found, and prepares the Output returned to the calling agent.

Flow Element	Role
Start	Starts the auto-launched flow.
Get Records	Finds the Parcel record using Parcel_ID__c and the input ids.
Decision	Checks whether the record was found.

Output / Assignment	Prepares the tracking summary.
End	Returns the response to the caller.

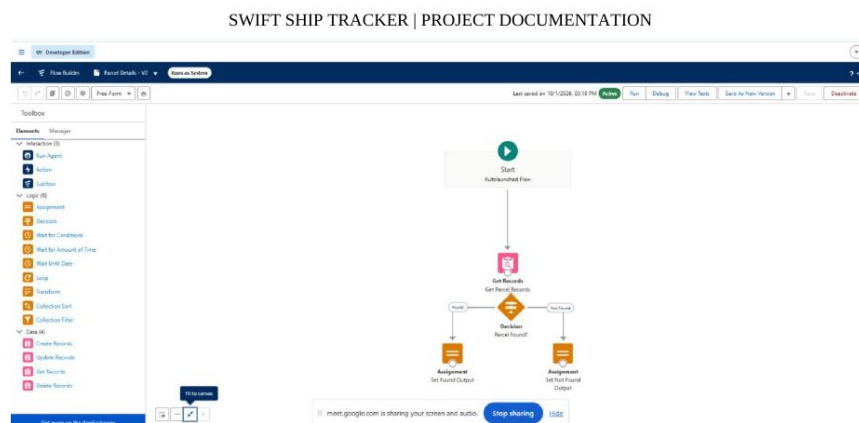


Figure 6.8.1 - Parcel Details Flow in Flow Builder.

6.9 Milestone 7 - Agentforce Parcel Tracking

Agentforce provides the conversational tracking layer. The configured Parcel Tracking capability can invoke the parcel-details action so that a user can ask for a parcel using its ID.

1. Enable Agentforce in the Salesforce Developer Org.
2. Open the agent in Agentforce Builder.
3. Configure the parcel tracking topic / subagent.
4. Add the Parcel Details Flow as an agent action

Figure 6.8.1 - Parcel Details Flow in Flow Builder.

6.9 Milestone 7 - Agentforce Parcel Tracking

Agentforce provides the conversational tracking layer. The configured Parcel Tracking capability can invoke the parcel-details action so that a user can ask for a parcel using its ID.

- Enable Agentforce in the Salesforce Developer Org.
- Open the agent in Agentforce Builder.
- Configure the parcel tracking topic / subagent.
- Add the Parcel Details Flow as an agent action.
- Map ids to the Parcel ID supplied by the user.
- Map Output to the agent response.
- Save and activate the agent.
- Test the agent using a known parcel ID.

5. Map the response to the variables supplied by the agent.
6. Map Output to the agent response.
7. Save and activate the agent.
8. Test the agent using a known parcel ID.

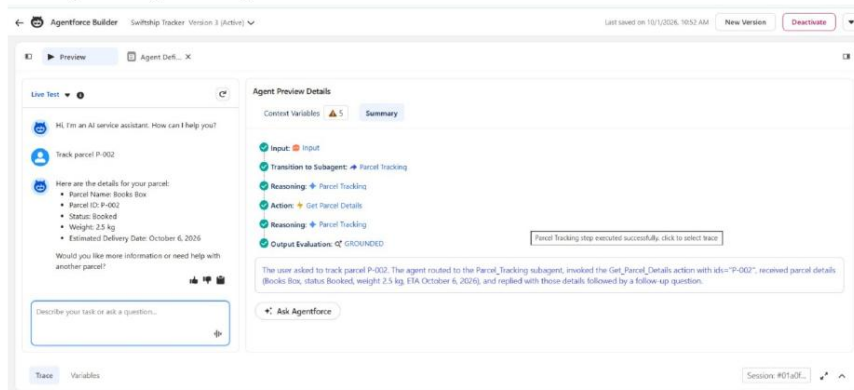


Figure 6.9.1 - Agentforce parcel tracking test for P-002.

Figure 6.9.1

- Agentforce parcel tracking test for P-002.

6.10 Source Control & Deployment

The project is stored as a Salesforce DX source project and pushed to the Swift-Ship-Tracker GitHub repository. The repository contains Salesforce metadata, scripts, deployment helpers, README documentation and project screenshots.

Component	Location
Salesforce metadata	force-app/main/default/*
Configuration	config/
Sample data	scripts/apex/seed.apex
Screenshots	screenshots/
Documentation	README.md
Windows deployment	deploy.cmd
macOS/Linux deployment	deploy.sh
Project definition	sfdx-project.json

Deployment Commands

```
sf project deploy start --source-dir force-app --target-org swiftship --wait 30
sf org assign permset --name Swift_Ship --target-org swiftship
sf apex run --file scripts/apex/seed.apex --target-org swiftship
sf org open --target-org swiftship
```

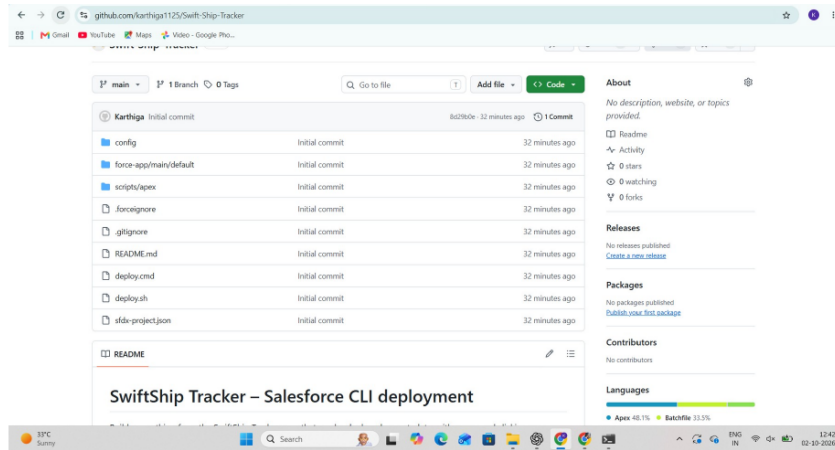


Figure 6.10.1 - GitHub repository containing SwiftShip Tracker project source.

6.11 Phase 4 - Data Migration, Testing & Security

The current project uses an Apex seed script for demonstration data. Testing focuses on verifying that Salesforce objects, Flow execution, Prompt Builder configuration, Agentforce interaction, permissions and deployment work together in the Developer Org. Production UAT, sandbox staging and enterprise CI/CD are outside the current prototype scope.

Figure 6.10.1 - GitHub repository containing SwiftShip Tracker project source.

6.11 Phase 4 - Data Migration, Testing & Security

The current project uses an Apex seed script for demonstration data. Testing focuses on verifying that Salesforce objects, Flow execution, Prompt Builder configuration, Agentforce interaction, permissions and deployment work together in the Developer Org. Production UAT, sandbox staging and enterprise CI/CD are outside the current prototype scope.

6.12 Phase 5 - Deployment, Documentation & Maintenance

Documentation records the implementation, screenshots and deployment workflow. Ongoing maintenance should keep parcel and delivery data accurate, monitor failed Flow executions or agent interactions, update Flow logic when requirements change, refine prompts and review permission access.

7. FUNCTIONAL & PERFORMANCE TESTING

Testing was conducted in the Salesforce Developer Org to validate the implemented workflow. The main focus is correct topic recognition, Flow invocation, record retrieval and response generation.

Test ID	Test Case	Expected Result	Status
TC-01	Open Parcel object	Parcel custom object is available.	Pass
TC-02	Open Delivery object	Delivery custom object is available.	Pass
TC-03	Check Parcel ID	Auto-number follows P-{000} format.	Pass
TC-04	Run Parcel Details Flow	Matching parcel record is retrieved.	Pass

TC-05	Prompt Builder	Parcel details are structured for response.	Pass
TC-06	Agentforce - Track P-002	Parcel details are returned in the agent preview.	Pass
TC-07	Permission set	Required Swift Ship access is available.	Pass
TC-08	Apex seed script	Demonstration records are created.	Pass
TC-09	GitHub repository	Project source and documentation are available.	Pass

7.1 Agentforce Validation

The captured Agentforce test shows a user request to track parcel P-002. The agent routes the request to parcel tracking, invokes the Get Parcel Details action, retrieves the parcel information and returns the parcel name, ID, status, weight and estimated delivery date.

TC-09	GitHub repository	Project source and documentation are available.	Pass
-------	-------------------	---	------

7.1 Agentforce Validation

The captured Agentforce test shows a user request to track parcel P-002. The agent routes the request to parcel tracking, invokes the Get Parcel Details action, retrieves the parcel information and returns the parcel name, ID, status, weight and estimated delivery date.

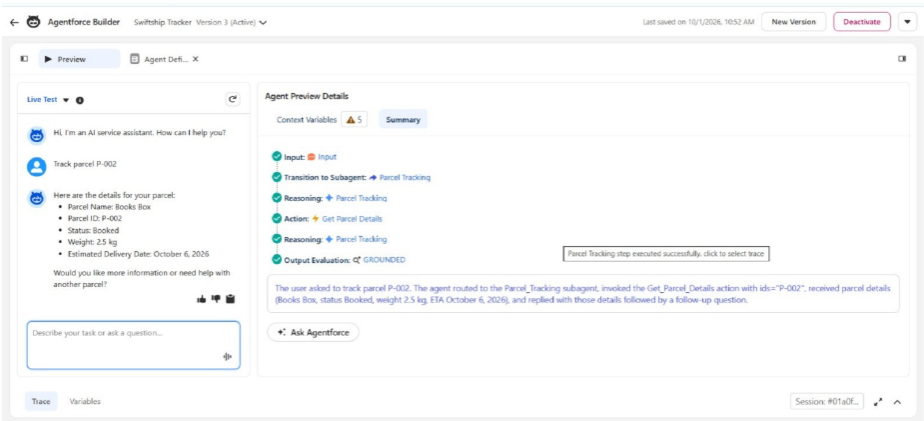


Figure 7.1 - Agentforce functional validation evidence.

Figure 7.1 - Agentforce functional validation evidence.

7.2 Performance Discussion

For an academic Developer Org prototype, testing emphasizes successful execution and response correctness. Formal load testing, production performance benchmarking and enterprise-scale availability testing are not part of the current implementation.

8. ADVANTAGES & DISADVANTAGES

Advantages

- Centralized parcel and delivery information.
- Conversational parcel tracking through Agentforce.
- Automated parcel retrieval using Salesforce Flow.
- Prompt Builder support for structured tracking responses.
- Reusable permission set and deployable metadata.
- GitHub-based source control and documentation.
- Clear path for future logistics features.

Disadvantages / Current Limitations

- The solution depends on Salesforce configuration and org capabilities.
- Agentforce behavior depends on correct prompts, actions, Flow configuration and available data.
- Email alerts and templates are not included in the current package.
- Batch Apex is not included in the current implementation.
- Reports and dashboards are not included in the current implementation.
- Experience Cloud customer portal is not included in the current implementation.
- Production UAT, sandbox staging and CI/CD are outside the current prototype scope.

9. CONCLUSION

SwiftShip Tracker demonstrates how Salesforce can be used to build a centralized parcel tracking workflow. The project integrates custom objects, relationships, Flow automation, Prompt Builder, Agentforce, permissions, Apex sample data and Salesforce CLI deployment.

The implemented Agentforce test provides a conversational method for retrieving parcel information from Salesforce using a Parcel ID. The project also demonstrates the benefit of maintaining Salesforce configuration in source control so the solution can be reviewed and redeployed.

The current system is suitable as an academic/prototype implementation and provides a foundation for additional logistics capabilities such as notifications, analytics, customer self-service and external integrations.

10. FUTURE SCOPE

- Add automated email and SMS alerts for booking and status changes.
- Add reports and dashboards for parcel and delivery performance.
- Build an Experience Cloud customer tracking portal.
- Extend Agentforce to support additional parcel-management questions and actions.
- Add Apex and Batch Apex automation for larger operational workloads.
- Integrate carrier, mapping or external logistics APIs.
- Introduce sandbox-based UAT, CI/CD and enterprise deployment controls.
- Improve prompts and conversational experiences, including multilingual support.
- Add stronger monitoring and audit capabilities.

10.1 Maintenance Roadmap

- Review parcel and delivery data quality regularly.

- Monitor Flow and Agentforce execution errors.
- Refine prompts when user queries evolve.
- Update permission access when new metadata is introduced.
- Maintain GitHub history and deployment scripts.
- Validate changes in a test environment before production use.

11. APPENDIX

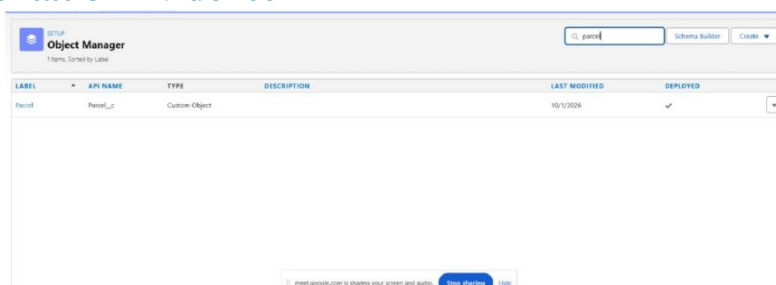
A. Custom Objects

- Parcel__c
- Delivery__c
- Sender__c
- Receiver__c

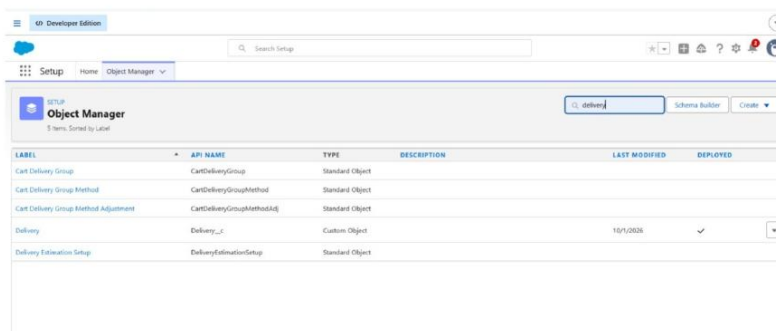
B. Key Metadata

Item	Location
Objects	force-app/main/default/objects/*
Tabs	force-app/main/default/tabs/*
Application	applications/SwiftShip Tracker
Flow	flows/Parcel_Details
Permission Set	permissionsets/Swift_Ship
Apex Seed	scripts/apex/seed.apex

C. Implementation Evidence



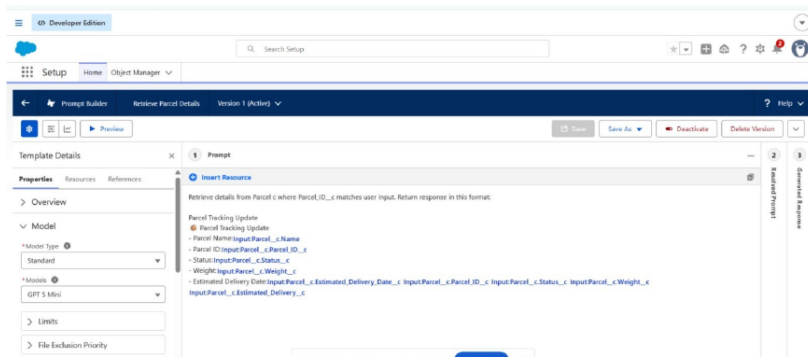
Appendix Figure C.1 - Parcel object.



Appendix Figure C.1 - Parcel object.

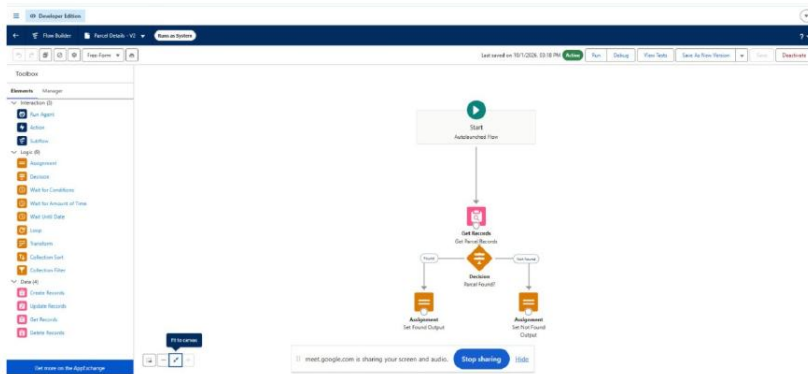
SWIFT SHIP TRACKER | PROJECT DOCUMENTATION

Appendix Figure C.2 - Delivery object.



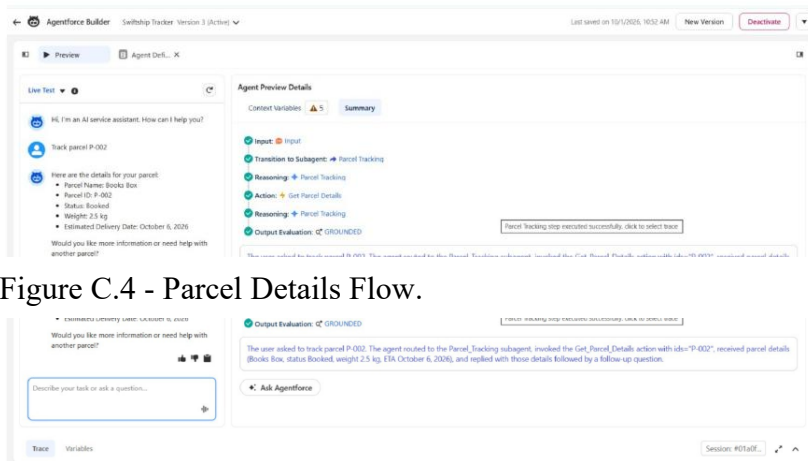
Appendix Figure C.2 - Delivery object.

Appendix Figure C.3 - Prompt Builder.



Appendix Figure C.3 - Prompt Builder.

Appendix Figure C.4 - Parcel Details Flow.



Appendix Figure C.4 - Parcel Details Flow.

Appendix Figure C.5 - Agentforce tracking test.

Appendix Figure C.5 - Agentforce tracking test.

12. REFERENCES

- Salesforce Developer Edition - project development and testing platform.
- Salesforce CLI - authentication and metadata deployment.
- Salesforce Object Manager - custom objects, fields and relationships.
- Salesforce Flow Builder - automation and record retrieval.
- Salesforce Prompt Builder - prompt-based information formatting.
- Salesforce Agentforce - conversational AI service-agent configuration.
- Git and GitHub - project source control and documentation repository.
- Swift Ship Tracker sample documentation provided for this project - used as the requested report structure and terminology reference.

Project Repository

GitHub: <https://github.com/karthiga1125/Swift-Ship-Tracker>